

EduManager Student Management System

Basic project data, working explanation, database structure, setup steps, and data creation guide for presentation.

Project Folder: C:\Users\Yash\Downloads\student mangment

Backend: Python FastAPI + SQLAlchemy

Database: PostgreSQL first, SQLite fallback

Prepared For: Project explanation to sir

Frontend: Angular 17 + Bootstrap

API Port: 3001, **Frontend Port:** 4200

1. Project Overview

EduManager is a student management system for managing schools, users, students, teachers, courses, attendance, grades, and analytics. The project has two main parts: a backend API that stores and returns data, and a frontend web app that users operate in the browser.

Frontend

Angular screens for login, dashboard, schools, students, teachers, courses, attendance, grades, and analytics.

Backend

FastAPI REST API receives requests, validates data, performs database operations, and returns JSON.

Database

Stores schools, users, student profiles, teacher profiles, courses, attendance, and marks.

2. Main Technologies

Layer	Technology	Use
Frontend	Angular 17, TypeScript, Bootstrap, Bootstrap Icons	Builds browser UI and calls backend APIs.
Backend	FastAPI, Uvicorn	Runs REST API on <code>http://localhost:3001</code> .
Database ORM	SQLAlchemy	Maps Python classes to database tables.
Validation	Pydantic	Checks request and response data format.
Authentication	JWT, python-jose, passlib bcrypt	Login token creation and password hashing.
Database	PostgreSQL or SQLite	PostgreSQL is tried first; if unavailable, SQLite file <code>sms_db.sqlite</code> is used.

3. Folder Structure

```
C:\Users\Yash\Downloads\student mangment
|-- backend
|   |-- main.py           FastAPI app entry point
|   |-- database.py       Database connection and session setup
|   |-- models.py         SQLAlchemy table models
|   |-- schemas.py        Pydantic request/response schemas
|   |-- auth_utils.py     Password hashing and JWT helpers
|   |-- seed.py           Creates sample school, users, teacher, student, course
|   |-- create_user.py    Example script for creating one student
|   |-- routers           API modules
|   |-- sms_db.sqlite     SQLite database file
|   |-- requirements.txt  Python dependencies
|   |-- run_server.bat    Starts backend server
|
|-- frontend
|   |-- src/app/pages     Main Angular pages
|   |-- src/app/core      API service, auth service, guards, interceptors
|   |-- src/environments  API URL configuration
|   |-- package.json      Frontend scripts and dependencies
```

4. Backend Working

The backend starts from `backend/main.py`. It creates database tables using `models.Base.metadata.create_all(bind=engine)`, enables CORS for Angular, and includes API routers.

Router	API Prefix	Purpose
auth.py	/api/auth	Login and registration.
schools.py	/api/schools	Create, list, update, and deactivate schools.
users.py	/api/users	Create and manage user accounts.
students.py	/api/students	Create and manage student profiles.
teachers.py	/api/teachers	Create and manage teacher profiles.
courses.py	/api/courses	Create and manage courses.
attendance.py	/api/attendance	Mark and view attendance.
grades.py	/api/grades	Add and view marks/grades.
analytics.py	/api/analytics	Dashboard counts and school reports.

5. Frontend Working

The frontend is an Angular app. It calls the API URL defined in `frontend/src/environments/environment.ts`: `http://localhost:3001/api`. Routes are defined in `src/app/app.routes.ts`.

Page Route	Who Can Access	Use
/login	Everyone	User login.
/dashboard	Logged-in users	Summary cards and quick actions.
/schools	Super admin	Manage schools.
/students	Super admin, school admin, teacher	Manage students.
/teachers	Super admin, school admin	Manage teachers.
/courses	Logged-in users	Manage or view courses.
/attendance	Logged-in users	Mark and view attendance.
/grades	Logged-in users	Add and view grades.
/analytics	Super admin, school admin	View reports and statistics.

6. Database Tables

Table	Important Fields	Relationship
schools	name, address, city, email, phone, subscription_plan, is_active	One school has many users, students, teachers, and courses.
users	email, password_hash, role, first_name, last_name, phone, school_id	User can be admin, teacher, student, parent, or super admin.
students	school_id, user_id, enrollment_number, grade_level, section, guardian details	Student profile links to one user and one school.
teachers	school_id, user_id, employee_id, department, qualification, salary	Teacher profile links to one user and one school.
courses	school_id, teacher_id, name, code, grade_level, section, schedule	Course belongs to a school and can be assigned to a teacher.
attendance	student_id, course_id, date, status, remarks	Attendance links student and course.
grades	student_id, course_id, exam_name, marks_obtained, total_marks, grade_letter	Grade links student and course.

7. User Roles

Role	Meaning	Example Access
super_admin	Main system admin	Manage schools and view full analytics.
school_admin	Admin for one school	Manage students, teachers, courses, school analytics.
teacher	Teacher user	View/manage students, courses, attendance, grades.
student	Student user	View student dashboard and school information.
parent	Parent role available in model	Can be extended for parent dashboard.

8. How To Run The Project

Backend

1. Open terminal in `C:\Users\Yash\Downloads\student mangment\backend`.
2. Install dependencies if needed: `pip install -r requirements.txt`.
3. Run backend: `run_server.bat`.
4. Backend API will run at `http://localhost:3001`.
5. API documentation opens at `http://localhost:3001/docs`.

Frontend

1. Open another terminal in `C:\Users\Yash\Downloads\student mangment\frontend`.
2. Install packages if needed: `npm install`.
3. Start Angular: `npm start` or `ng serve`.
4. Open browser: `http://localhost:4200`.

Start backend first, then frontend. The frontend expects the API at `http://localhost:3001/api`.

9. Default Sample Data

The file `backend/seed.py` creates basic demo data. Run it from the backend folder:

```
python seed.py
```

Data	Value Created
School	Delhi Public School, Dwarka, New Delhi, email info@dps.edu.in, premium plan.
Super Admin	admin@edumanager.com / admin123
School Admin	principal@dps.edu.in / school123
Teacher	anita.sharma@dps.edu.in / teacher123, Mathematics department, employee ID TCH-001.
Student	rahul.verma@dps.edu.in / student123, enrollment STU-2026-001, class 10-A.
Course	Mathematics, code MATH-10, class 10-A, schedule Mon/Wed/Fri 9:00-10:00.

10. How To Create Data In The App

Create School

1. Login as super admin.
2. Open **Schools**.
3. Click add/create school.
4. Enter school name, city, email, phone, website, plan, and save.

Create Student

1. Login as super admin or school admin.
2. Open **Students**.
3. Add student details: school, first name, last name, email, phone, grade level, section, guardian details, and enrollment number.
4. System creates a user account with role **student** and a linked student profile.

Create Teacher

1. Login as super admin or school admin.
2. Open **Teachers**.
3. Enter school, name, email, phone, employee ID, department, qualification, specialization, joining date, and salary.
4. System creates a user account with role **teacher** and a linked teacher profile.

Create Course

1. Open **Courses**.
2. Enter school, course name, code, teacher, grade level, section, description, and schedule.
3. Save the course so attendance and grades can use it.

Mark Attendance

1. Open **Attendance**.
2. Select student, course, date, and status: present, absent, late, or excused.
3. Save records. The backend stores them in the **attendance** table.

Add Grades

1. Open `Grades`.
2. Select student and course.
3. Enter exam name, marks obtained, total marks, grade letter, remarks, and date.
4. Save. The analytics page can calculate average grades from this data.

11. API Examples For Explanation

These examples can be shown in FastAPI Swagger at <http://localhost:3001/docs>.

```
POST /api/auth/login
{
  "email": "admin@edumanager.com",
  "password": "admin123"
}
```

```
POST /api/schools
{
  "name": "ABC Public School",
  "city": "Ahmedabad",
  "state": "Gujarat",
  "email": "info@abcschool.edu.in",
  "phone": "9876543210",
  "subscription_plan": "basic"
}
```

```
POST /api/students
{
  "school_id": 1,
  "user_id": 1,
  "email": "student.personal@gmail.com",
  "first_name": "Yash",
  "last_name": "Patel",
  "password": "student123",
  "grade_level": "10",
  "section": "A",
  "enrollment_number": "STU-2026-003",
  "guardian_name": "Parent Name",
  "guardian_phone": "9876543210"
}
```

```
POST /api/attendance
{
  "records": [
    {
      "student_id": 1,
      "course_id": 1,
      "date": "2026-05-25",
      "status": "present",
      "remarks": "On time"
    }
  ]
}
```

```
]
}
```

```
POST /api/grades
{
  "student_id": 1,
  "course_id": 1,
  "exam_name": "Unit Test 1",
  "marks_obtained": 85,
  "total_marks": 100,
  "grade_letter": "A",
  "remarks": "Good performance",
  "date": "2026-05-25"
}
```

12. Login And Authentication Flow

1. User enters email and password on Angular login page.
2. Angular sends data to `POST /api/auth/login`.
3. Backend checks the user email and verifies bcrypt password hash.
4. If correct, backend creates a JWT token with user email and role.
5. Frontend stores the token and uses route guards to protect pages.
6. Role guard controls which page each role can open.

13. How API Is Connected And Data Is Fetched

In this project, Angular does not talk to the database directly. Angular calls the FastAPI backend, and the backend talks to the database. This is the main flow:

```
Angular Page Component
-> calls Angular ApiService/AuthService function
-> HttpClient sends request to http://localhost:3001/api/...
-> FastAPI router function receives request
-> SQLAlchemy queries or inserts database data
-> Backend returns JSON response
-> Angular subscribe() receives JSON
-> Component variable updates and HTML table/card displays data
```

API Base URL

The frontend API connection is stored in `frontend/src/environments/environment.ts`. This file tells Angular where the backend is running.

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:3001/api',
};
```

If backend is running on another port, change this `apiUrl`. For this project, backend port is `3001`, so frontend calls URLs like `http://localhost:3001/api/students`.

Main API Service File

Most API functions are written in `frontend/src/app/core/api.service.ts`. This file uses Angular `HttpClient` to call backend APIs.

```
private base = environment.apiUrl;

getStudents(params?: any): Observable<any> {
  let httpParams = new HttpParams();
  // query params are added here
  return this.http.get(`${this.base}/students`, { params: httpParams });
}

createStudent(data: any): Observable<any> {
  return this.http.post(`${this.base}/students`, data);
}
```

Meaning: `getStudents()` fetches students using GET request, and `createStudent()` creates a new student using POST request.

Where Function Is Called In Frontend

Example: in `frontend/src/app/pages/students/students.component.ts`, the `load()` function calls `this.api.getStudents(params)`. When the response comes, it stores data into `this.students`.

```
load() {
  const params: any = { search: this.search || undefined, page: this.pagination.page };

  this.api.getStudents(params).subscribe({
    next: r => {
      this.students = r.students || [];
      this.pagination = r.pagination || this.pagination;
    },
    error: () => {},
  });
}
```

Then the HTML table uses `*ngFor="let s of students"` to display all student rows.

```
<tr *ngFor="let s of students">
  <td>{{ s.firstName }} {{ s.lastName }}</td>
  <td>{{ s.enrollmentNumber }}</td>
  <td>{{ s.section }}</td>
</tr>
```

How Create Student Works

When user clicks save in the student form, `onSave()` runs. If editing is false, it calls `createStudent()`. After successful save, it hides the form and calls `load()` again to refresh table data.

```
onSave() {
  const obs = this.editing
    ? this.api.updateStudent(this.editing.id, this.form)
    : this.api.createStudent(this.form);

  obs.subscribe({
```

```

    next: () => {
      this.showForm = false;
      this.load();
    },
    error: () => {},
  });
}

```

Backend Function That Receives Student API

The backend student APIs are in `backend/routers/students.py`. The route prefix is `/api/students`. So this function handles `GET /api/students`.

```

@router.get("")
def list_students(..., db: Session = Depends(get_db)):
    query = db.query(models.Student)
    students = query.offset(offset).limit(limit).all()
    return {
        "students": [_student_to_dict(s) for s in students],
        "pagination": {
            "page": page,
            "totalPages": total_pages,
            "total": total
        }
    }

```

This function reads from the `students` table using SQLAlchemy and returns JSON.

Backend Function That Creates Student

This function handles `POST /api/students`. It creates a user first, then creates a student profile linked with that user.

```

@router.post("", status_code=201)
def create_student(data: schemas.StudentCreate, db: Session = Depends(get_db)):
    user = models.User(
        email=official_email,
        password_hash=hashed,
        role=models.RoleEnum.student,
        first_name=data.first_name,
        last_name=data.last_name,
        school_id=data.school_id,
    )
    db.add(user)
    db.commit()
    db.refresh(user)

    student = models.Student(
        school_id=data.school_id,
        user_id=user.id,
        enrollment_number=data.enrollment_number,
        grade_level=data.grade_level,
        section=data.section,
    )
    db.add(student)
    db.commit()

```

```
db.refresh(student)
return _student_to_dict(student)
```

14. Login API Code Flow

Login starts from `frontend/src/app/pages/login/login.component.ts`. When user submits the form, `onLogin()` calls `this.auth.login(email, password)`.

```
onLogin(): void {
  this.auth.login(this.email, this.password).subscribe({
    next: () => this.router.navigate(['/dashboard']),
    error: (err) => this.error = 'Login failed.'
  });
}
```

The login API function is in `frontend/src/app/core/auth.service.ts`.

```
login(email: string, password: string): Observable<LoginResponse> {
  return this.http.post<LoginResponse>(`${this.apiUrl}/auth/login`, { email, password }).pipe(
    tap(res => {
      localStorage.setItem('sms_token', res.accessToken);
      localStorage.setItem('sms_user', JSON.stringify(res.user));
      this.currentUserSubject.next(res.user);
    })
  );
}
```

Backend receives this in `backend/routers/auth.py` at `POST /api/auth/login`. It checks email, verifies password, creates JWT token, and returns user data.

```
@router.post("/login")
def login(request: schemas.LoginRequest, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.email == request.email).first()
    if not user:
        raise HTTPException(status_code=401, detail="Invalid credentials")

    if not auth_utils.verify_password(request.password, user.password_hash):
        raise HTTPException(status_code=401, detail="Invalid credentials")

    access_token = auth_utils.create_access_token(
        data={"sub": user.email, "role": user.role}
    )
    return { "accessToken": access_token, "user": {...} }
```

15. Common Function Call Map

User Action	Angular Component Function	Angular Service Function	Backend API Function
Login	<code>LoginComponent.onLogin()</code>	<code>AuthService.login()</code>	<code>auth.py login()</code>
Load Dashboard Counts	<code>DashboardComponent.ngOnInit()</code>	<code>getOverviewAnalytics()</code>	<code>analytics.py overview_analytics()</code>
Load Students	<code>StudentsComponent.load()</code>	<code>getStudents()</code>	<code>students.py list_students()</code>
Add Student	<code>StudentsComponent.onSave()</code>	<code>createStudent()</code>	<code>students.py create_student()</code>
Edit Student	<code>StudentsComponent.onSave()</code>	<code>updateStudent()</code>	<code>students.py update_student()</code>
Delete Student	<code>StudentsComponent.onDelete()</code>	<code>deleteStudent()</code>	<code>students.py delete_student()</code>
Load Teachers	<code>TeachersComponent.load()</code>	<code>getTeachers()</code>	<code>teachers.py list_teachers()</code>
Add Teacher	<code>TeachersComponent.onSave()</code>	<code>createTeacher()</code>	<code>teachers.py create_teacher()</code>
Load Courses	<code>CoursesComponent load method</code>	<code>getCourses()</code>	<code>courses.py list_courses()</code>
Mark Attendance	<code>AttendanceComponent save method</code>	<code>markAttendance()</code>	<code>attendance.py mark_attendance()</code>
Add Grade	<code>GradesComponent save method</code>	<code>createGrade()</code>	<code>grades.py create_grade()</code>

16. How To Create Similar Code Yourself

1. Create database model in `models.py` , for example `Student` .
2. Create request/response schema in `schemas.py` , for example `StudentCreate` .
3. Create backend router function in `routers/students.py` , for example `@router.post("")` .
4. Include router in `main.py` using `app.include_router(students.router)` .
5. Create Angular service function in `api.service.ts` , for example `createStudent(data)` .
6. Call service function from component, for example `StudentsComponent.onSave()` .
7. Bind HTML form fields using `[(ngModel)]` so user input goes into `form` object.
8. Use `subscribe()` to receive response and refresh the page data.

```
// Simple frontend create pattern
form = { name: '', email: '' };
```

```
onSave() {  
  this.api.createSomething(this.form).subscribe({  
    next: () => this.load(),  
    error: () => alert('Error saving data')  
  });  
}  
  
// Simple backend create pattern  
@router.post("")  
def create_item(data: ItemCreate, db: Session = Depends(get_db)):  
    item = models.Item(**data.model_dump())  
    db.add(item)  
    db.commit()  
    db.refresh(item)  
    return item
```

17. How To Explain API Fetching To Sir

"Sir, the frontend and backend are connected through REST APIs. I stored the backend URL in the Angular environment file. Then I created an ApiService file where all HTTP functions are written. For example, when I open the Students page, the component's load function calls getStudents from ApiService. ApiService sends a GET request to /api/students. FastAPI receives it in students.py, queries the database using SQLAlchemy, and returns JSON. Angular receives that JSON inside subscribe and assigns it to the students array. The HTML table uses ngFor to display the array. For create operations, the form data is sent by POST request, backend inserts it into database, and frontend reloads the list."

18. Explanation Script For Sir

"Sir, this is my EduManager student management project. I have divided it into two parts: Angular frontend and FastAPI backend. The frontend gives screens like login, dashboard, schools, students, teachers, courses, attendance, grades, and analytics. The backend provides REST APIs and stores data using SQLAlchemy models. First, the user logs in. The backend verifies the password and returns a JWT token. According to the user's role, the frontend allows access to different pages. For example, the super admin can manage schools, school admin can manage school data, teachers can work with students and academic data, and students can view their dashboard. The main database tables are schools, users, students, teachers, courses, attendance, and grades. I also created a seed file to generate demo data for quick testing."

19. Important Notes

- If PostgreSQL is not available, the backend automatically falls back to SQLite.
- The SQLite database file is `backend/sms_db.sqlite`.
- Use `http://localhost:3001/docs` to test APIs directly.
- Use `http://localhost:4200` to use the Angular application.
- Seed login accounts are useful for demo and viva explanation.